

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)**

Факультет **Прикладной информатики**

Образовательная программа **Мобильные и облачные технологии**

Направление подготовки **09.03.03 Прикладная информатика**

О Т Ч Е Т

по ЛАБОРАТОРНОЙ РАБОТЕ №4

**«Запросы на выборку и модификацию данных. Представления.
Работа с индексами» по дисциплине «Проектирование и
реализация баз данных»**

Вариант свой: Учёт выполнения заданий

Выполнили: **Абакар Иссака Малли К3241**

Преподаватель : **Говорова Марина Михайловна**

Дата **06.05.2026**

Санкт-Петербург 2026

Цель работы

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Оборудование

Компьютерный класс.

Программное обеспечение

СУБД PostgreSQL, pgAdmin 4.

Практическое задание

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов.
3. Изучить графическое представление запросов и просмотреть историю запросов.
4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

Схема базы данных

Для выполнения лабораторной работы использовалась база данных со схемой `company\_schema`, содержащая следующие таблицы:

1. departments
2. positions
3. employees
4. employee_history
5. organizations
6. contracts
7. projects
8. tasks
9. participation
10. control

11. invoices
12. project_employees
13. project_payments

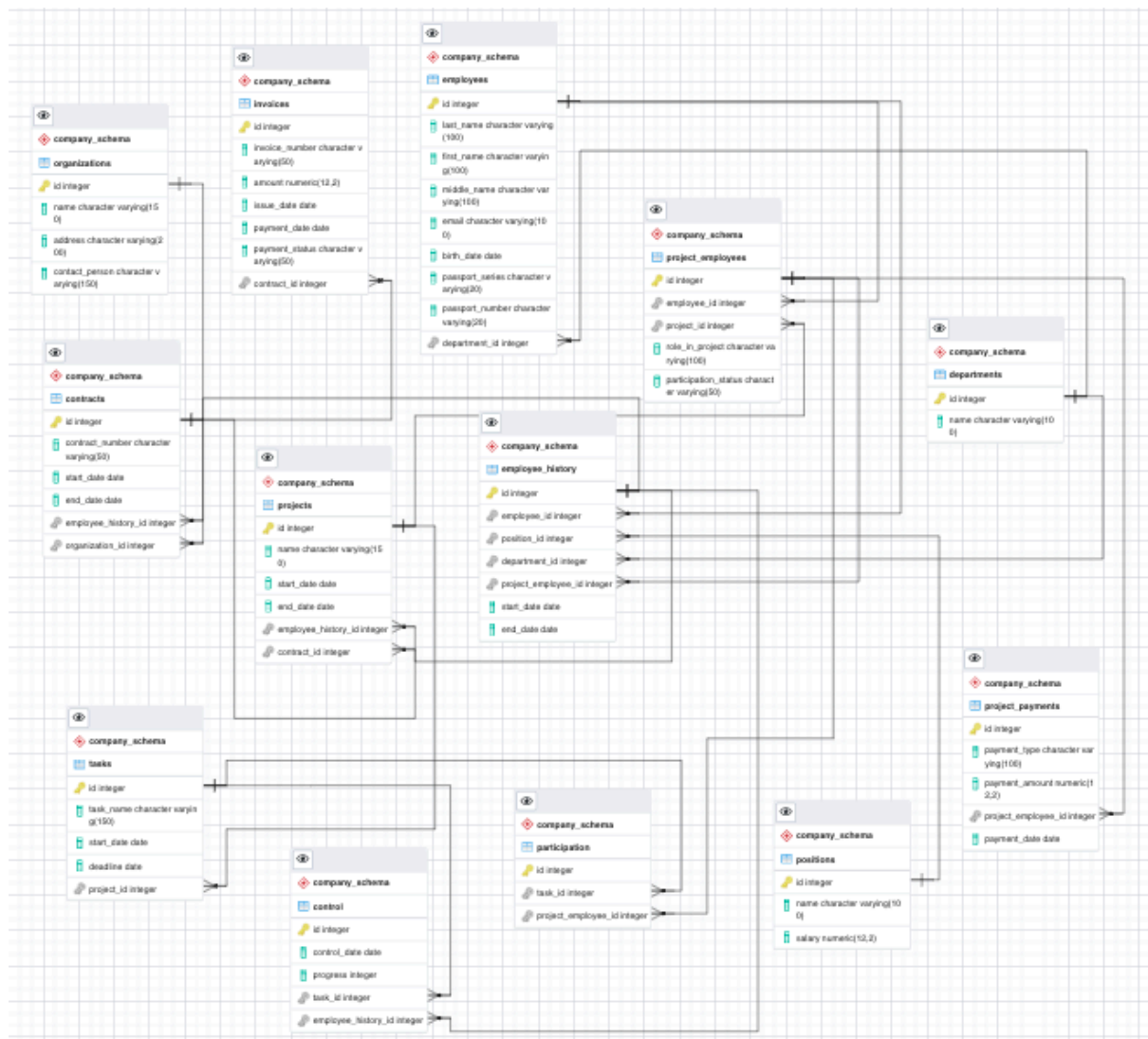


Рисунок 1 – Схема логической модели базы данных, сгенерированная в Generate ERD

1. Выполнение SELECT-запросов

1.1 Список сотрудников с указанием отдела

```
SELECT e.last_name,
       e.first_name,
       d.name AS department

FROM company_schema.employees e
JOIN company_schema.departments d
ON e.department_id = d.id;
```

Данный запрос объединяет таблицы `employees` и `departments` и выводит фамилию, имя сотрудника и название отдела.

Query Editor

Query History

1

SELECT e.last_name,

2

e.first_name,

3

d.name AS department

4

FROM company_schema.employees e

5

JOIN company_schema.departments d

6

ON e.department_id = d.id;

.

Data Output

Explain

Messages

Notifications

last_name

character varying (100)

first_name

character varying (100)

department

character varying (100)

1.2 Список проектов и соответствующих договоров

```
SELECT p.name AS project_name,  
       c.contract_number  
FROM company_schema.projects p  
JOIN company_schema.contracts c  
ON p.contract_id = c.id;
```

Запрос показывает название проекта и номер договора, на основании которого он выполняется.

Query Editor	Query History
<pre>1 SELECT p.name AS project_name, 2 c.contract_number 3 FROM company_schema.projects p 4 JOIN company_schema.contracts c 5 ON p.contract_id = c.id;</pre>	
Data Output	Explain Messages Notifications
 project_name character varying (150)	 contract_number character varying (50)

1.3 Список счетов и статусов оплаты

```

SELECT invoice_number,
       amount,
       payment_status
FROM company_schema.invoices;

```

Запрос выводит номер счета, сумму и текущий статус оплаты.

Query Editor		Query History	
1	SELECT	invoice_number,	
2		amount,	
3		payment_status	
4	FROM	company_schema.invoices;	
5			

Data Output		Explain	Messages	Notifications
	invoice_number	amount	payment_status	
	character varying (50)	numeric (12,2)	character varying (50)	

2. Создание представления (VIEW)

2.1 Создание представления `v\_employees\_departments`

```

CREATE OR REPLACE VIEW company_schema.v_employees_departments
AS
SELECT e.last_name,
       e.first_name,
       d.name AS department
FROM company_schema.employees e
JOIN company_schema.departments d
ON e.department_id = d.id;

```

Созданное представление объединяет информацию о сотрудниках и отделах.

Query Editor

Query History

```

1 CREATE OR REPLACE VIEW company_schema.v_employees_departments AS
2 SELECT e.last_name,
3        e.first_name,
4        d.name AS department
5 FROM company_schema.employees e
6 JOIN company_schema.departments d
7 ON e.department_id = d.id;

```

Data Output

Explain

Messages

Notifications

CREATE VIEW

Query returned successfully in 275 msec.

2.2 Проверка представления

```
SELECT * FROM company_schema.v_employees_departments;
```

8

9

```

8 SELECT * FROM company_schema.v_employees_departments;
9

```

Data Output

Explain

Messages

Notifications

	last_name character varying (100) 	first_name character varying (100) 	department character varying (100) 
---	--	---	---

3. Операция INSERT с подзапросом

```

INSERT INTO company_schema.tasks
(task_name, start_date, deadline, project_id)
VALUES (
    'Documentation',
    CURRENT_DATE,
    CURRENT_DATE + INTERVAL '7 days',
    (SELECT id

```

```

        FROM company_schema.projects
        LIMIT 1)
);

```

Подзапрос выбирает идентификатор первого проекта, который используется при добавлении новой задачи.

```

1  INSERT INTO company_schema.tasks
2  (task_name, start_date, deadline, project_id)
3  VALUES (
4      'Documentation',
5      CURRENT_DATE,
6      CURRENT_DATE + INTERVAL '7 days',
7      (SELECT id
8       FROM company_schema.projects
9       LIMIT 1)
10 );
11
12

```

Data Output Explain **Messages** Notifications

INSERT 0 1

Query returned successfully in 342 msec.

Проверка

```
SELECT * FROM company_schema.tasks;
```

```

11
12  SELECT * FROM company_schema.tasks;
13

```

Data Output Explain Messages Notifications

	id [PK] integer	task_name character varying (150)	start_date date	deadline date	project_id integer
1	1	Documentation	2026-05-12	2026-05-19	[null]

4. Операция UPDATE с подзапросом

Запрос изменяет отдел сотрудника Петрова, используя подзапрос для поиска идентификатора отдела.

Query returned successfully in 275 msec.

	id	last_name	first_name	middle_name	email	birth_date	passport_series	passport_number
	[PK] integer	character varying (100)	character varying (100)	character varying (100)	character varying (100)	date	character varying (20)	character varying (20)

5. Операция DELETE с подзапросом

```
DELETE FROM company_schema.tasks
WHERE project_id = (
    SELECT id
    FROM company_schema.projects
    WHERE name = 'Project Beta'
)
AND task_name = 'Testing';
```

Запрос удаляет задачу `Testing`, относящуюся к проекту `Project Beta`.

```
1 DELETE FROM company_schema.tasks
2 WHERE project_id = (
3     SELECT id
4     FROM company_schema.projects
5     WHERE name = 'Project Beta'
6 )
7 AND task_name = 'Testing';
8
9 |
```

Data Output	Explain	Messages	Notifications
-------------	---------	----------	---------------

DELETE 0

Query returned successfully in 327 msec.

Проверка

```
SELECT * FROM company_schema.tasks;
```

```
9 SELECT * FROM company_schema.tasks;
```

Data Output

Explain

Messages

Notifications

	id [PK] integer	task_name character varying (150)	start_date date	deadline date	project_id integer
1	1	Documentation	2026-05-12	2026-05-19	[null]

6. Создание индекса

```
CREATE INDEX idx_employees_last_name  
ON company_schema.employees(last_name);
```

Индекс создается по полю `last\_name` для ускорения поиска сотрудников по фамилии.

```
1 CREATE INDEX idx_employees_last_name  
2 ON company_schema.employees(last_name);  
3 |
```

Data Output	Explain	Messages	Notifications
CREATE INDEX			
Query returned successfully in 289 msec.			

7. Анализ выполнения запроса с EXPLAIN ANALYZE

```
EXPLAIN ANALYZE
```

```
SELECT *  
FROM company_schema.employees  
WHERE last_name = 'Ivanov';
```

Команда `EXPLAIN ANALYZE` показывает план выполнения запроса и время его выполнения.

После создания индекса выполнение запроса становится более эффективным.

```
1 EXPLAIN ANALYZE  
2 SELECT *  
3 FROM company_schema.employees  
4 WHERE last_name = 'Ivanov';  
5 |
```

Data Output

Explain

Messages

Notifications

1	QUERY PLAN text	🔒
1	Index Scan using idx_employees_last_name on employees (cost=0.14..8.16 rows=1 width=1000) (actual time=0.042..0.043 rows=0 loops=1)	
2	[...] Index Cond: ((last_name)::text = 'Ivanov'::text)	
3	Planning Time: 41.680 ms	
4	Execution Time: 40.647 ms	

8. История выполненных запросов

В pgAdmin была просмотрена вкладка `Query History`, содержащая список всех выполненных SQL-команд.

Show queries generated internally by pgAdmin?

Yes

Today - 12/05/2026

▶ EXPLAIN ANALYZE SELECT * FROM company_schema.employee...
12:52:30

▶ CREATE INDEX idx_employees_last_name ON company_schem...
12:50:21

▶ SELECT * FROM company_schema.tasks;
12:48:58

▶ DELETE FROM company_schema.tasks WHERE project_id = (...
12:48:15

▶ SELECT * FROM company_schema.employees;
12:45:18

▶ UPDATE company_schema.employees SET department_id = (...

[Data Output](#) [Explain](#) [Messages](#) [Notifications](#)

Вывод

В ходе выполнения лабораторной работы были успешно освоены основные приемы работы с SQL в PostgreSQL. Были составлены запросы `SELECT`, создано представление, выполнены операции `INSERT`, `UPDATE` и `DELETE` с использованием подзапросов. Также был создан индекс для ускорения поиска данных и проведен анализ выполнения запроса с помощью `EXPLAIN ANALYZE`.

Полученные результаты показали, что использование индексов позволяет повысить производительность запросов, а представления упрощают работу с часто используемыми выборками данных.

Таким образом, цель лабораторной работы была полностью достигнута.

Приложение: Список использованных SQL-команд

1. SELECT
2. JOIN
3. CREATE VIEW
4. INSERT INTO ... SELECT
5. UPDATE ... SET (...)
6. DELETE ... WHERE (...)
7. CREATE INDEX
8. EXPLAIN ANALYZE